

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

Báo Cáo Nhập môn An toàn thông tin

Đề tài: **Các cơ chế phát hiện xâm nhập mạng**

Học phần: **Nhập môn An toàn thông tin**

Mã học phần: IT4015

Giảng viên hướng dẫn: PGS. TS. Nguyễn Linh Giang

Sinh viên thực hiện:	Nguyễn Đình Tuấn Minh	20230048
	Vũ Đức Tâm	20230064
	Nguyễn Gia Khánh	202416803
	Nguyễn Đăng Cao Tuấn	202400119

Hà Nội, June 24, 2026

Mục lục

Lời nói đầu	3
1 Giới thiệu đề tài	4
1.1 Bối cảnh và lý do chọn đề tài	4
1.2 Mục tiêu đề tài	4
1.3 Đối tượng và phạm vi nghiên cứu	4
1.4 Phương pháp thực hiện	5
1.5 Bố cục báo cáo	5
2 Cơ sở lý thuyết	6
2.1 IDS, IPS và NIDS	6
2.2 Phát hiện theo chữ ký và Chỉ số thỏa hiệp (IOC)	7
2.3 Thuật toán so khớp chuỗi trong NIDS	8
2.3.1 Thuật toán Boyer-Moore (So khớp đơn mẫu)	8
2.3.2 Thuật toán Aho-Corasick (So khớp đa mẫu)	8
2.3.3 Vấn đề quay lui của Biểu thức chính quy (PCRE) và ReDoS	9
2.4 Phát hiện theo hành vi và Cửa sổ trượt	9
2.5 Các kiểu tấn công được mô hình hóa trong bài tập lớn	10
2.5.1 Quét cổng (Port Scanning)	10
2.5.2 Tấn công từ chối dịch vụ ICMP Flood	10
2.5.3 Tấn công từ chối dịch vụ TCP SYN Flood	10
2.5.4 Kênh truyền độc hại qua DNS (DNS Tunneling)	11
2.6 Giới hạn lý thuyết của việc triển khai NIDS	11
3 Kiến trúc và Triển khai Hệ thống NIDS	13
3.1 Tổng quan kiến trúc	13
3.2 Thu thập và chuẩn hóa gói tin	13
3.2.1 Capture	13
3.2.2 PacketEvent	14
3.3 Detection Engine	14
3.4 Luật phát hiện tích hợp	15
3.4.1 Cơ chế chống tấn công cạn kiệt bộ nhớ (State Exhaustion DoS)	15
3.5 Luật chữ ký dạng Suricata-style subset	16
3.6 Lưu trữ và Dashboard hiển thị	16
3.6.1 Lưu trữ cảnh báo dạng Append-Only JSON Lines (JSONL)	16
3.6.2 Cơ chế hiển thị trên Dashboard Flask	17
4 Kịch bản Thử nghiệm và Đánh giá	18
4.1 Môi trường thử nghiệm	18
4.2 Khởi động hệ thống	18
4.3 Các kịch bản kiểm thử	19

4.3.1	Quét cổng bằng nmap	19
4.3.2	ICMP Ping Flood	19
4.3.3	TCP SYN Flood	19
4.3.4	DNS tunneling	19
4.3.5	DNS signature rule	20
4.4	Phân tích cấu trúc nhật ký cảnh báo sinh ra (JSONL)	20
4.4.1	Cảnh báo hành vi: Quét cổng (Port Scan)	20
4.4.2	Cảnh báo hành vi: ICMP Ping Flood	20
4.4.3	Cảnh báo hành vi: TCP SYN Flood	21
4.4.4	Cảnh báo bất thường: DNS Tunneling	21
4.5	Đánh giá kết quả	22
5	Kết luận và Hướng phát triển	23
5.1	Kết quả đạt được	23
5.2	Hạn chế	23
5.3	Hướng phát triển	24
5.4	Kết luận	24

Lời nói đầu

Trong kỷ nguyên số hóa, hệ thống mạng là nền tảng cho hầu hết hoạt động của tổ chức và doanh nghiệp. Cùng với lợi ích đó, các rủi ro an ninh mạng cũng xuất hiện thường xuyên hơn, đặc biệt là các hành vi thăm dò, khai thác dịch vụ và truyền dữ liệu bất thường qua những giao thức hợp lệ.

Trong bối cảnh đó, các biện pháp phòng thủ tĩnh như tường lửa vẫn cần thiết nhưng chưa đủ để quan sát nội dung và hành vi của lưu lượng mạng. Hệ thống phát hiện xâm nhập mạng (NIDS) giúp bổ sung lớp giám sát, tạo cảnh báo khi lưu lượng có dấu hiệu bất thường hoặc khớp với mẫu tấn công đã biết.

Trong khuôn khổ học phần **IT4015 - Nhập môn An toàn thông tin**, nhóm em lựa chọn thực hiện đề tài: **"Các cơ chế phát hiện xâm nhập mạng"**. Trọng tâm của bài tập lớn là xây dựng một NIDS nhỏ gọn bằng Python để minh họa quy trình thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký và hiển thị cảnh báo.

Mục tiêu của báo cáo là trình bày ngắn gọn cơ sở lý thuyết cần thiết, mô tả kiến trúc hệ thống đã triển khai, kiểm thử trong môi trường lab và chỉ ra các giới hạn còn tồn tại.

Mặc dù đã nỗ lực tìm hiểu và tổng hợp tài liệu, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, bản báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm em rất mong nhận được sự nhận xét và góp ý quý báu từ thầy để kiến thức của nhóm chúng em được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 24 tháng 06 năm 2026

Nhóm sinh viên thực hiện

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và lý do chọn đề tài

Trong các hệ thống mạng hiện đại, tường lửa chỉ là một lớp bảo vệ ban đầu. Nhiều cuộc tấn công vẫn đi qua các cổng hợp lệ như HTTP, DNS hoặc HTTPS, sau đó khai thác điểm yếu ở tầng ứng dụng hoặc tạo ra các hành vi bất thường trong mạng nội bộ. Vì vậy, hệ thống phát hiện xâm nhập mạng (Network Intrusion Detection System – NIDS) là một thành phần quan trọng để giám sát lưu lượng, phát hiện dấu hiệu tấn công và hỗ trợ quản trị viên phản ứng kịp thời.

Đề tài này tập trung xây dựng một NIDS nhỏ gọn bằng Python nhằm minh họa các thành phần cốt lõi của một hệ thống phát hiện xâm nhập: thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký, lưu trữ cảnh báo và hiển thị qua dashboard. Mục tiêu chính là phục vụ học tập và trình diễn kỹ thuật, không hướng tới thay thế các hệ thống sản xuất như Suricata hoặc Zeek.

1.2 Mục tiêu đề tài

Bài tập đặt ra các mục tiêu sau:

- **Thu thập và chuẩn hóa gói tin:** đọc lưu lượng từ tệp PCAP hoặc giao diện mạng trực tiếp bằng Scapy, sau đó chuyển đổi thành cấu trúc sự kiện thống nhất.
- **Phát hiện theo hành vi:** triển khai các luật dựa trên cửa sổ thời gian để nhận diện quét cổng, ICMP flood, TCP SYN flood và dấu hiệu DNS tunneling.
- **Phát hiện theo chữ ký/IOC:** xây dựng bộ phân tích cú pháp cho một tập con cú pháp kiểu Suricata và kiểm tra các mẫu IOC như domain blacklist hoặc truy vấn DNS đã biết.
- **Ghi nhận và trực quan hóa cảnh báo:** lưu cảnh báo dưới dạng JSON Lines và hiển thị thống kê qua giao diện Flask.
- **Kiểm thử trong môi trường lab:** mô phỏng các luồng tấn công phổ biến bằng Kali Linux để đánh giá khả năng kích hoạt cảnh báo của hệ thống.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là hệ thống phát hiện xâm nhập mạng dạng thụ động. Hệ thống quan sát lưu lượng, tạo cảnh báo và phục vụ phân tích, nhưng không trực tiếp chặn gói tin hoặc ngắt phiên kết nối như một hệ thống IPS.

Phạm vi triển khai của bài tập lớn được giới hạn như sau:

- Hệ thống xử lý các trường thông tin phổ biến của IP, TCP, UDP, ICMP và DNS.
- Phần phát hiện hành vi sử dụng ngưỡng tĩnh và cửa sổ trượt thời gian; các ngưỡng này là giá trị lab/demo, cần hiệu chỉnh theo baseline từng mạng thực tế.
- Phần chữ ký hỗ trợ một *Suricata-style subset*, đủ cho tình huống demo khớp domain IOC mẫu trong truy vấn DNS.
- Hệ thống chưa giải mã TLS/HTTPS, chưa tái tạo đầy đủ TCP stream và chưa được tối ưu cho lưu lượng mạng tốc độ cao.

1.4 Phương pháp thực hiện

Đề tài được triển khai theo hướng kết hợp lý thuyết và thực nghiệm:

- Nghiên cứu các cơ chế IDS cơ bản, đặc biệt là phát hiện theo chữ ký và phát hiện theo hành vi.
- Thiết kế kiến trúc pipeline gồm các khối capture, parser, detection engine, storage và dashboard.
- Cài đặt từng module bằng Python, sử dụng Scapy cho xử lý gói tin và Flask cho giao diện web.
- Viết luật kiểm thử và kịch bản tấn công mô phỏng để kiểm tra kết quả cảnh báo.
- Đối chiếu kết quả với phạm vi đã đặt ra, đồng thời chỉ ra hạn chế còn tồn tại.

1.5 Bố cục báo cáo

Báo cáo gồm 5 chương:

- **Chương 1: Giới thiệu đề tài** – trình bày bối cảnh, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương 2: Cơ sở lý thuyết** – tóm tắt các khái niệm IDS/IPS, NIDS, phát hiện theo chữ ký và phát hiện theo hành vi.
- **Chương 3: Kiến trúc và triển khai hệ thống** – mô tả các module chính và cách dữ liệu đi qua hệ thống.
- **Chương 4: Kịch bản thử nghiệm và đánh giá** – trình bày môi trường lab, các kịch bản tấn công và kết quả quan sát.
- **Chương 5: Kết luận và hướng phát triển** – tổng kết kết quả đạt được, hạn chế và hướng cải tiến.

Chương 2

Cơ sở lý thuyết

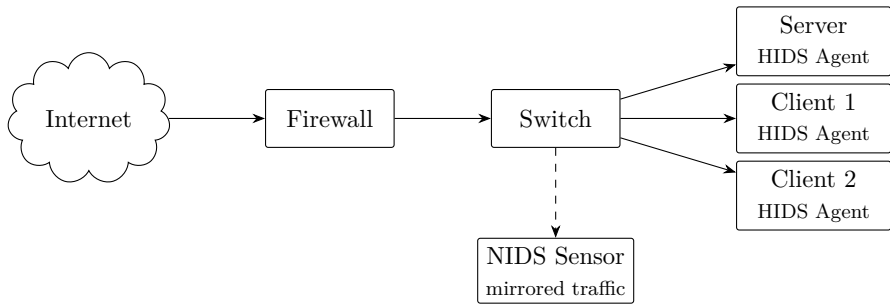
2.1 IDS, IPS và NIDS

Hệ thống phát hiện xâm nhập (Intrusion Detection System – IDS) là một giải pháp giám sát an ninh mạng có nhiệm vụ thu thập, phân tích thông tin từ các nguồn khác nhau trong hệ thống CNTT để phát hiện hành vi vi phạm chính sách bảo mật hoặc các dấu hiệu tấn công. IDS hoạt động theo cơ chế thụ động (passive mode): quan sát dữ liệu đi qua, ghi nhật ký (logging) và phát cảnh báo (alerting) đến quản trị viên mà không trực tiếp thay đổi hoặc can thiệp vào luồng gói tin thực tế.

Ngược lại với IDS, Hệ thống ngăn chặn xâm nhập (Intrusion Prevention System – IPS) hoạt động ở chế độ chủ động (active mode). IPS thường được triển khai nằm trực tiếp trên đường truyền (inline). Khi phát hiện gói tin độc hại hoặc lưu lượng bất thường, IPS có khả năng can thiệp trực tiếp bằng cách hủy gói tin (drop), ngắt kết nối (reset TCP), hoặc cấu hình lại tường lửa để chặn IP nguồn của kẻ tấn công ngay lập tức. Trong phạm vi bài tập lớn này, nhóm tập trung nghiên cứu và triển khai theo hướng IDS để tối ưu hóa hiệu năng giám sát thụ động.

Dựa trên nguồn dữ liệu và vị trí giám sát, IDS được chia thành hai trường phái chính:

- **NIDS (Network-based IDS):** Giám sát toàn bộ lưu lượng mạng đi qua một hoặc nhiều phân đoạn mạng (network segments). NIDS thu thập dữ liệu bằng cách lắng nghe trực tiếp thông qua card mạng đặt ở chế độ hỗn tạp (promiscuous mode), hoặc qua cổng cấu hình Span/Mirror port trên Switch, hoặc thiết bị TAP vật lý. Ưu điểm của NIDS là tính độc lập cao, không làm ảnh hưởng đến hiệu năng của các máy chủ dịch vụ, và có khả năng giám sát đồng thời nhiều thiết bị trong mạng.
- **HIDS (Host-based IDS):** Cài đặt trực tiếp dưới dạng các tác nhân (agents) trên từng hệ điều hành máy chủ hoặc máy trạm (endpoint). HIDS thu thập dữ liệu từ log hệ thống, log ứng dụng, tính toàn vẹn của các file cấu hình quan trọng (file integrity monitoring - FIM), các cuộc gọi hệ thống (system calls) và các tiến trình đang chạy. HIDS có ưu điểm lớn trong việc phát hiện các cuộc tấn công leo thang đặc quyền nội bộ hoặc các gói tin độc hại đã được giải mã ngay tại host, nhưng đòi hỏi tài nguyên cài đặt và quản lý phức tạp trên diện rộng.



Hình 2.1: Minh họa vị trí giám sát của NIDS và HIDS

Loại hệ thống	Nguồn dữ liệu	Đặc điểm
NIDS	Lưu lượng mạng tại switch, gateway hoặc interface giám sát.	Quan sát được nhiều máy trong cùng phân đoạn mạng, phù hợp phát hiện scan, flood và payload độc hại trong lưu lượng không mã hóa.
HIDS	Log, tiến trình, file và hành vi trên từng máy chủ hoặc endpoint.	Nhìn sâu vào trạng thái máy trạm nhưng cần cài agent trên từng host.

Bảng 2.1: Phân loại IDS theo nguồn dữ liệu giám sát

Hệ thống trong bài tập là một NIDS dạng thụ động: đọc PCAP hoặc sniff interface, phân tích gói tin và ghi cảnh báo ra tệp JSONL.

2.2 Phát hiện theo chữ ký và Chỉ số thỏa hiệp (IOC)

Phát hiện theo chữ ký (Signature-based Detection) là phương pháp đối sánh dữ liệu lưu lượng mạng với một tập hợp các mẫu (signatures) hoặc luật (rules) đã được định nghĩa trước về hành vi độc hại. Các chữ ký này thường được trích xuất từ các Chỉ số thỏa hiệp (Indicators of Compromise – IOC) bao gồm: chuỗi nhị phân đặc trưng của mã độc (malware signature), địa chỉ IP thuộc mạng botnet, tên miền độc hại của máy chủ điều khiển (C2 server), hoặc các mẫu chuỗi ký tự trong giao thức ứng dụng.

Ưu điểm lớn nhất của phương pháp này là tốc độ xử lý nhanh đối với các cuộc tấn công đã biết, tỷ lệ cảnh báo sai (false positive) cực thấp và cung cấp thông tin chi tiết về loại tấn công giúp kiểm chứng dễ dàng. Tuy nhiên, điểm hạn chế chí mạng là hoàn toàn không thể phát hiện các cuộc tấn công chưa từng được công bố (Zero-day) hoặc các biến thể mã độc đã được làm rối (obfuscation) hoặc mã hóa lưu lượng.

Trong dự án này, nhóm đã tự triển khai một công cụ phân tích cú pháp (parser) và nhân luật (engine) tương thích với định dạng luật của Suricata/Snort. Một luật chuẩn bao gồm:

- **Header (Phần đầu):** Khai báo hành động (ví dụ: `alert`), giao thức mạng (ví dụ: `tcp`, `udp`, `http`), dải địa chỉ IP nguồn/đích, cổng dịch vụ nguồn/đích và hướng của luồng dữ liệu (`->` một chiều, hoặc `<>` hai chiều).
- **Rule Options (Phần tùy chọn):** Được bao quanh bởi dấu ngoặc đơn, chứa các từ khóa điều khiển:
 - `msg`: Chuỗi mô tả thông tin cảnh báo xuất ra log.

- **content**: Chuỗi ký tự nhị phân hoặc văn bản cần tìm kiếm trong gói tin (hỗ trợ modifier như **nocase** - không phân biệt hoa thường).
- **pcre**: Biểu thức chính quy tương thích Perl giúp khớp các mẫu phức tạp hơn.
- **dns.query**: Từ khóa chỉ định vị trí đối sánh thuộc phần truy vấn của giao thức DNS.
- **http.uri**, **http.method**, **http.header**: Các buffer tách biệt chứa dữ liệu HTTP để tăng hiệu năng tìm kiếm thay vì tìm trên toàn bộ gói tin thô.
- **sid** (Signature ID): Định danh duy nhất cho luật đó.

Ví dụ một luật mẫu phát hiện truy vấn tên miền phục vụ kiểm thử độc hại:

```
alert dns any any -> any 53 (msg:"Custom DNS Query: example.com";  
dns.query; content:"example.com"; nocase; sid:1000101; rev:1;)
```

2.3 Thuật toán so khớp chuỗi trong NIDS

Trong các hệ thống NIDS dựa trên chữ ký, nhiệm vụ quan trọng nhất và tiêu tốn nhiều tài nguyên CPU nhất (thường chiếm tới 70% – 80% thời gian xử lý gói tin) là tìm kiếm hàng ngàn chuỗi ký tự (signatures) độc hại trong phần payload của hàng triệu gói tin đi qua mỗi giây. Để giải quyết bài toán này, các hệ thống IDS/IPS hiện đại như Suricata hay Snort không sử dụng phương pháp duyệt tuần tự thông thường mà áp dụng các thuật toán so khớp chuỗi tối ưu hóa cao:

2.3.1 Thuật toán Boyer-Moore (So khớp đơn mẫu)

Thuật toán Boyer-Moore là thuật toán tìm kiếm chuỗi đơn hiệu quả nhất trong thực tế. Khác với thuật toán tìm kiếm thông thường (so khớp từ trái sang phải), Boyer-Moore thực hiện so khớp các ký tự từ phải sang trái của mẫu (pattern) nhưng dịch chuyển mẫu từ trái sang phải trên chuỗi văn bản cần tìm kiếm (text). Thuật toán tận dụng thông tin thu được từ các ký tự không khớp thông qua hai bảng dịch chuyển:

- **Bad Character Rule (Quy tắc ký tự xấu)**: Khi xảy ra lỗi không khớp tại ký tự $T[i] \neq P[j]$, thuật toán sẽ tìm kiếm ký tự $T[i]$ trong mẫu P . Nếu có, dịch chuyển mẫu để ký tự $T[i]$ trong mẫu trùng khớp với $T[i]$ trong văn bản. Nếu không có, dịch chuyển toàn bộ mẫu vượt qua vị trí $T[i]$.
- **Good Suffix Rule (Quy tắc hậu tố tốt)**: Nếu một phần hậu tố của mẫu khớp với văn bản trước khi xảy ra lỗi không khớp, thuật toán sẽ dịch chuyển mẫu đến vị trí tiếp theo mà hậu tố này (hoặc một tiền tố của mẫu) xuất hiện lại trong mẫu.

Nhờ hai quy tắc này, Boyer-Moore cho phép bỏ qua rất nhiều ký tự trong văn bản mà không cần kiểm tra, đạt độ phức tạp thời gian tốt nhất là $O(n/m)$ với n là độ dài văn bản và m là độ dài mẫu.

2.3.2 Thuật toán Aho-Corasick (So khớp đa mẫu)

Khi số lượng chữ ký tăng lên hàng chục ngàn, việc chạy thuật toán đơn mẫu như Boyer-Moore cho từng chữ ký trên mỗi gói tin sẽ gây ra suy giảm hiệu năng nghiêm trọng (độ phức tạp tỉ lệ thuận với số lượng chữ ký). Thuật toán Aho-Corasick giải quyết vấn đề này bằng cách xây dựng một máy trạng thái hữu hạn (Trie-based Finite State Automaton) chứa tất cả các chữ ký cần tìm kiếm. Máy trạng thái Aho-Corasick hoạt động dựa trên ba hàm cốt lõi:

- **Goto function (Hàm chuyển trạng thái)**: Dẫn dắt quá trình di chuyển từ trạng thái này sang trạng thái khác dựa trên ký tự tiếp theo trong văn bản. Nó tạo ra một cấu trúc cây phân cấp (Trie) cho tất cả các mẫu.

- **Failure function (Hàm thất bại):** Khi một ký tự không khớp với bất kỳ nhánh con nào tại trạng thái hiện tại, hàm thất bại sẽ chỉ ra trạng thái tiếp theo cần chuyển đến (thường là hậu tố dài nhất của chuỗi hiện tại cũng là tiền tố của một mẫu khác trong cây). Hàm này giúp thuật toán không cần phải quay lui (backtrack) trên chuỗi văn bản đầu vào.
- **Output function (Hàm đầu ra):** Liên kết các trạng thái với danh sách các mẫu được phát hiện tương ứng khi trạng thái đó được tiếp cận.

Nhờ cơ chế này, thuật toán Aho-Corasick có thể tìm kiếm đồng thời tất cả các chữ ký trong văn bản chỉ với một lần duyệt duy nhất. Độ phức tạp thời gian đạt mức tuyến tính $O(n + z)$ trong đó n là độ dài văn bản và z là số lần xuất hiện của các mẫu độc hại, hoàn toàn độc lập với số lượng chữ ký cài đặt. Đây là nền tảng của cơ chế quét `fast_pattern` trong Suricata.

2.3.3 Vấn đề quay lui của Biểu thức chính quy (PCRE) và ReDoS

Mặc dù so khớp chuỗi tĩnh rất nhanh, kẻ tấn công có thể dễ dàng thay đổi một vài ký tự trong payload để né tránh. Do đó, các IDS cần dùng biểu thức chính quy (PCRE) để mô tả các biến thể tấn công linh hoạt hơn. Tuy nhiên, việc sử dụng các công cụ regex truyền thống dựa trên NFA (Nondeterministic Finite Automaton) có khả năng gây ra hiện tượng **Quay lui thảm khốc (Catastrophic Backtracking)**. Khi gặp một chuỗi đầu vào không khớp được thiết kế đặc biệt (ví dụ: khớp biểu thức $(a^+)^+$ với chuỗi `aaaaaaaaaaaaaaaaaaaaaaaaaaaaaab`), động cơ regex sẽ thử tất cả các tổ hợp quay lui có thể, dẫn đến độ phức tạp thời gian tăng theo cấp số nhân $O(2^n)$. Kẻ tấn công có thể lợi dụng lỗ hổng này để thực hiện tấn công từ chối dịch vụ vào chính hệ thống IDS (Regular Expression Denial of Service – ReDoS). Vì lý do này, các IDS hiện đại thường sử dụng công cụ regex của Google (RE2) hoạt động dựa trên DFA (Deterministic Finite Automaton) để đảm bảo thời gian chạy tuyến tính, hoặc hạn chế nghiêm ngặt việc sử dụng các mẫu regex có cấu trúc lặp lồng nhau trong các file luật.

2.4 Phát hiện theo hành vi và Cửa sổ trượt

Khác với phát hiện theo chữ ký tập trung vào một gói tin đơn lẻ, Phát hiện theo hành vi (Behavior-based Detection) tập trung phân tích xu hướng hoặc mối liên hệ giữa các gói tin theo chuỗi thời gian. Phương pháp này đặc biệt hiệu quả trong việc nhận diện các kiểu tấn công do thám (reconnaissance) hoặc từ chối dịch vụ (DoS/DDoS) vốn không mang nội dung độc hại trong từng gói tin riêng lẻ nhưng lại tạo ra lưu lượng bất thường khi gộp lại.

Để thực hiện điều này một cách tối ưu trong môi trường thời gian thực mà không làm tràn bộ nhớ, kỹ thuật **Cửa sổ trượt thời gian (Sliding Window)** được áp dụng.

- Hệ thống duy trì một hàng đợi hai đầu (double-ended queue - `deque`) lưu trữ dấu vết thời gian (timestamps) của các gói tin nhận được từ mỗi địa chỉ IP nguồn trong khoảng thời gian T giây (window size).
- Với mỗi gói tin mới đến từ một nguồn, hệ thống tiến hành loại bỏ các bản ghi cũ có $t < t_{\text{current}} - T$ ở đầu hàng đợi.
- Sau đó, thêm gói tin hiện tại vào cuối hàng đợi và kiểm tra xem tổng số gói tin hoặc số lượng thuộc tính duy nhất (ví dụ: các cổng đích khác nhau) có vượt quá ngưỡng N quy định hay không. Nếu có, phát cảnh báo về hành vi bất thường.

Phương pháp này giúp phát hiện được các hành vi nguy hại mà không cần biết trước signature của cuộc tấn công, tuy nhiên thách thức lớn nhất là xác định ngưỡng N và T tối ưu cho từng hạ tầng mạng thực tế. Nếu đặt ngưỡng quá nhạy sẽ dẫn đến báo động giả hàng loạt (false positives) do lưu lượng hợp lệ của người dùng; ngược lại, nếu đặt ngưỡng quá cao sẽ làm mất tác dụng cảnh báo của hệ thống (false negatives).

2.5 Các kiểu tấn công được mô hình hóa trong bài tập lớn

Bài tập lớn thực hiện nghiên cứu, mô phỏng và phát hiện 4 dạng tấn công mạng phổ biến dưới góc độ lý thuyết giao thức:

2.5.1 Quét cổng (Port Scanning)

Quét cổng là giai đoạn do thám đầu tiên trong vòng đời tấn công. Kẻ tấn công gửi các gói tin thăm dò tới một dải cổng dịch vụ của mục tiêu nhằm xác định trạng thái cổng (Open, Closed, Filtered) và hệ điều hành của nạn nhân. Các kỹ thuật quét cổng phổ biến bao gồm:

- **TCP SYN Scan (Stealth/Half-open Scan):** Kẻ tấn công gửi gói tin với cờ SYN thiết lập. Nếu nhận được SYN-ACK từ mục tiêu, cổng đó đang mở; kẻ tấn công lập tức gửi gói RST để đóng kết nối thay vì hoàn thành cái bắt tay ba bước (ACK). Điều này giúp quét nhanh và tránh bị ghi log bởi các ứng dụng ở tầng trên.
- **TCP FIN Scan:** Gửi gói tin chỉ có cờ FIN. Theo đặc tả RFC 793, nếu cổng đóng, hệ điều hành sẽ phản hồi bằng gói RST; nếu cổng mở, hệ điều hành sẽ lơ gói tin đi.
- **TCP NULL Scan & Xmas Scan:** NULL Scan gửi gói tin không bật bất kỳ cờ TCP nào. Xmas Scan bật đồng thời các cờ FIN, PSH, và URG (như một cây thông Noel sáng đèn). Cả hai đều lợi dụng phản hồi RST của cổng đóng để xác định trạng thái cổng, vượt qua các bộ lọc tường lửa thô sơ chỉ kiểm tra cờ SYN.

Hệ thống phát hiện quét cổng bằng cách theo dõi số lượng cổng đích duy nhất (**unique ports**) mà một địa chỉ IP nguồn gửi yêu cầu đến trong vòng T giây với các tổ hợp cờ TCP tương ứng (S, F, rỗng, hoặc FPU).

2.5.2 Tấn công từ chối dịch vụ ICMP Flood

ICMP Flood (Ping Flood) là hình thức tấn công từ chối dịch vụ cổ điển. Kẻ tấn công gửi liên tục các gói tin truy vấn ICMP Echo Request (**Type 8**) tới địa chỉ IP của nạn nhân với tốc độ cực cao nhằm làm nghẽn băng thông đường truyền hoặc khiến CPU của thiết bị mạng bị quá tải do phải xử lý và phản hồi liên tục bằng gói tin ICMP Echo Reply (**Type 0**). Hệ thống phát hiện bằng cách đếm số lượng gói tin ICMP có **Type 8**, **13**, **17** từ một địa chỉ nguồn trong cửa sổ trượt. Việc loại trừ các gói tin ICMP Reply ra khỏi bộ đếm là cực kỳ quan trọng để tránh tình trạng báo động giả lên chính nạn nhân khi họ đang gửi phản hồi lại kẻ tấn công.

2.5.3 Tấn công từ chối dịch vụ TCP SYN Flood

TCP SYN Flood lợi dụng cơ chế hàng đợi kết nối (backlog queue) trong quá trình bắt tay ba bước của giao thức TCP.

1. Kẻ tấn công gửi liên tục các gói tin SYN yêu cầu kết nối với địa chỉ IP nguồn giả mạo (spoofed IP) hoặc không tồn tại.
2. Máy chủ nạn nhân tiếp nhận, đưa kết nối vào hàng đợi nửa mở (Half-open Connection Queue / Backlog Queue), chuyển sang trạng thái SYN-RECEIVED và phản hồi bằng gói SYN-ACK tới địa chỉ IP nguồn giả mạo đó.
3. Do địa chỉ IP nguồn là giả mạo, máy chủ sẽ không bao giờ nhận được gói tin phản hồi ACK cuối cùng từ kẻ tấn công để hoàn tất kết nối. Kết nối nửa mở này sẽ được lưu giữ trong hàng đợi cho đến khi hết thời gian chờ (timeout).

4. Khi kẻ tấn công gửi hàng vạn gói tin SYN liên tục, hàng đợi backlog của máy chủ sẽ nhanh chóng bị lấp đầy, khiến máy chủ từ chối mọi yêu cầu kết nối TCP hợp lệ khác từ người dùng thông thường.

NIDS phát hiện cuộc tấn công này bằng cách theo dõi tần suất các gói tin chỉ bắt cờ SYN mà không bắt cờ ACK nhằm tới một dịch vụ cụ thể trên máy chủ mục tiêu.

2.5.4 Kênh truyền độc hại qua DNS (DNS Tunneling)

DNS Tunneling là kỹ thuật đóng gói lưu lượng của giao thức khác (chẳng hạn như HTTP, SSH hoặc lệnh điều khiển botnet) vào trong các gói tin DNS hợp lệ nhằm vượt qua các thiết bị kiểm soát biên. Vì DNS là dịch vụ cốt lõi bắt buộc phải mở trên tường lửa để phân giải tên miền, kẻ tấn công có thể lợi dụng điều này để rò rỉ dữ liệu hoặc duy trì kênh kết nối ẩn (C2). Cơ chế hoạt động dựa trên cấu trúc phân cấp DNS: kẻ tấn công đăng ký một tên miền (ví dụ: `badsite.com`) và trở bản ghi Name Server (NS) về một máy chủ C2 do chúng kiểm soát. Khi mã độc gửi truy vấn phân giải tên miền phụ chứa dữ liệu mã hóa:

`data_chuyen_di.badsite.com`

Truy vấn này sẽ được chuyển tiếp qua các DNS Resolver hợp lệ trên Internet và cuối cùng gửi tới máy chủ C2 của kẻ tấn công. Máy chủ C2 sẽ trích xuất phần tên miền phụ để đọc dữ liệu và phản hồi lại mã độc dưới dạng các bản ghi DNS TXT, CNAME, hoặc A chứa lệnh điều khiển mới.

Hệ thống phát hiện DNS Tunneling thông qua các dấu hiệu bất thường của chuỗi truy vấn DNS:

- **Độ dài truy vấn (Query Length):** Tên miền phụ chứa dữ liệu đóng gói thường có độ dài rất lớn (gần ngưỡng giới hạn 253 ký tự của giao thức).
- **Độ dài nhãn lớn nhất (Max Label Length):** Mỗi nhãn phân tách bởi dấu chấm có độ dài lớn (gần giới hạn 63 ký tự).
- **Độ hỗn loạn thông tin (Shannon Entropy):** Các tên miền thông thường được đặt theo ngôn ngữ tự nhiên nên có tính lặp lại cao và độ hỗn loạn thấp. Ngược lại, dữ liệu mã hóa độc hại (thường dùng mã hóa Base64 hoặc Base32) có tính ngẫu nhiên rất lớn. Công thức toán học tính Entropy Shannon của một chuỗi ký tự S là:

$$H(S) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

Trong đó $P(x_i)$ là tần suất xuất hiện của ký tự thứ x_i trong chuỗi S . Một tên miền phụ có độ dài nhãn lớn và entropy vượt quá ngưỡng 3.8 liên tục sẽ bị hệ thống đưa vào diện nghi vấn DNS Tunneling.

2.6 Giới hạn lý thuyết của việc triển khai NIDS

Mặc dù NIDS là công cụ quan trọng, trên khía cạnh lý thuyết và thực tiễn triển khai vẫn tồn tại các giới hạn sau:

- **Mã hóa lưu lượng (Encryption Blindness):** Khi các giao thức chuyển dịch sang HTTPS (TLS 1.3), DoT (DNS over TLS), DoH (DNS over HTTPS), NIDS hoàn toàn không thể đọc được nội dung của gói tin (payload) để thực hiện đối sánh chữ ký nếu không có giải pháp giải mã trung gian (SSL/TLS Decryption).

- **Thiếu khả năng lắp ráp luồng dữ liệu (No Stream Reassembly):** Các cuộc tấn công ứng dụng (như SQL Injection) có thể được kẻ tấn công chia nhỏ payload qua nhiều phân đoạn TCP (TCP Segmentation hoặc IP Fragmentation). Nếu NIDS phân tích từng gói tin độc lập (stateless) mà không tái cấu trúc lại luồng dữ liệu TCP (stream reassembly), hệ thống sẽ không phát hiện ra mã độc.
- **Kỹ thuật né tránh (Evasion Techniques):** Kẻ tấn công có thể gửi các gói tin có thời gian sống (TTL) khác nhau hoặc chồng chéo dữ liệu (fragment overlapping) khiến NIDS nhận dạng một đăng nhưng hệ điều hành của nạn nhân lại xử lý một nẻo, dẫn đến việc bỏ lọt tấn công.
- **Hiệu năng phần cứng và ngôn ngữ:** Việc sử dụng các ngôn ngữ bậc cao như Python kết hợp thư viện Scapy rất tốt cho việc nghiên cứu thử nghiệm học thuật, tuy nhiên trong môi trường mạng doanh nghiệp thực tế với băng thông lớn (Gigabit trở lên), hệ thống sẽ gặp hiện tượng nghẽn cổ chai CPU và rơi gói tin (packet dropping) hàng loạt. Các hệ thống thực tế phải sử dụng C/C++/Rust kết hợp các cơ chế bắt gói tin hiệu năng cao như DPDK hoặc PF_RING.

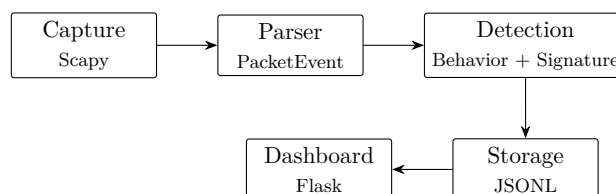
Chương 3

Kiến trúc và Triển khai Hệ thống NIDS

3.1 Tổng quan kiến trúc

Hệ thống NIDS được xây dựng bằng Python 3 và tổ chức theo mô hình pipeline. Mỗi gói tin sau khi được đọc từ PCAP hoặc interface mạng sẽ đi qua các bước: thu thập, chuẩn hóa, phát hiện, lưu trữ và hiển thị.

1. **Capture:** đọc gói tin bằng Scapy từ tệp PCAP hoặc giao diện mạng.
2. **Parser:** chuyển gói tin Scapy thành đối tượng `PacketEvent`.
3. **Detection Engine:** chạy luật hành vi/anomaly và luật chữ ký.
4. **Storage:** ghi cảnh báo ra tệp JSONL.
5. **Dashboard:** đọc dữ liệu cảnh báo và hiển thị thống kê bằng Flask.



Hình 3.1: Luồng dữ liệu của hệ thống NIDS

3.2 Thu thập và chuẩn hóa gói tin

3.2.1 Capture

Module `nids/capture.py` cung cấp hai cách lấy dữ liệu: đọc từ tệp PCAP và sniff trực tiếp trên interface. Với live capture, Scapy gọi callback cho từng gói tin; nếu parser chuyển đổi thành công thì sự kiện sẽ được đưa vào detection engine.

Đoạn mã 3.1: Live capture bằng Scapy

```
1 def sniff_live(interface: str, callback: PacketCallback, limit: int = 0) -> None:
2     from scapy.all import sniff
3
```

```
4 def handle_packet(packet: object) -> None:
5     event = packet_to_event(packet)
6     if event is not None:
7         callback(event)
8
9     sniff(iface=interface, prn=handle_packet, store=False, count=limit)
```

3.2.2 PacketEvent

Raw packet chứa nhiều trường phụ thuộc giao thức nên khó xử lý trực tiếp trong rule engine. Vì vậy, module `nids/parser.py` chuẩn hóa các trường cần thiết thành `PacketEvent`: thời gian, IP nguồn/đích, cổng nguồn/đích, giao thức, TCP flags, ICMP type, DNS query, payload text và các trường HTTP phổ biến.

Đoạn mã 3.2: Cấu trúc PacketEvent

```
1 @dataclass(frozen=True)
2 class PacketEvent:
3     timestamp: float
4     src_ip: str | None
5     dst_ip: str | None
6     src_port: int | None
7     dst_port: int | None
8     protocol: str
9     length: int
10    tcp_flags: str | None = None
11    icmp_type: int | None = None
12    dns_query: str | None = None
13    payload_text: str | None = None
14    http_method: str | None = None
15    http_uri: str | None = None
16    http_host: str | None = None
17    http_user_agent: str | None = None
```

Payload được giới hạn kích thước khi chuyển thành chuỗi để tránh tốn bộ nhớ và giảm chi phí so khớp. Đây là lựa chọn phù hợp cho demo, nhưng cũng là một giới hạn khi phân tích payload lớn hoặc payload bị chia nhỏ.

3.3 Detection Engine

Module `nids/engine.py` đóng vai trò điều phối. Mỗi `PacketEvent` được đếm vào thống kê giao thức, sau đó gửi đến `SlidingWindowRules`. Các cảnh báo sinh ra được ghi xuống `AlertStore`.

Đoạn mã 3.3: Luồng xử lý một sự kiện

```
1 def process_event(self, event: PacketEvent) -> list[Alert]:
2     self.packet_count += 1
3     self.protocol_counts[event.protocol] += 1
4
5     alerts = self.rules.evaluate(event)
6     self.store.append_many(alerts)
7     self.alert_count += len(alerts)
8     return alerts
```

3.4 Luật phát hiện tích hợp

Các luật hành vi và anomaly được triển khai bằng cửa sổ trượt theo thời gian. Hệ thống lưu các sự kiện gần nhất theo IP nguồn và loại bỏ dữ liệu đã nằm ngoài khung thời gian cấu hình. Các ngưỡng trong bảng là giá trị lab/demo, chưa phải khuyến nghị production.

Để lưu giữ vết thời gian (timestamps) của các gói tin trong cửa sổ trượt thời gian một cách tối ưu, hệ thống sử dụng hàng đợi hai đầu (`collections.deque`) thay vì danh sách thông thường (`list`).

- **Hiệu năng chèn và xóa:** Việc loại bỏ các gói tin cũ ở đầu hàng đợi đòi hỏi thao tác xóa ở đầu. Đối với Python `list`, thao tác `pop(0)` có độ phức tạp thời gian là $O(N)$ do phải dịch chuyển toàn bộ phần tử còn lại trong bộ nhớ. Đối với `deque`, thao tác `popleft()` chỉ mất thời gian $O(1)$, giúp hệ thống duy trì hiệu suất xử lý gói tin cực cao ở thời gian thực.
- **Thuật toán loại bỏ phần tử hết hạn:**

Đoạn mã 3.4: Loại bỏ sự kiện cũ trong cửa sổ thời gian

```
1 @staticmethod
2 def _drop_old_times(window: deque[float], now: float, seconds: int) -> None:
3     while window and now - window[0] > seconds:
4         window.popleft()
```

3.4.1 Cơ chế chống tấn công cạn kiệt bộ nhớ (State Exhaustion DoS)

Một trong những điểm yếu lớn nhất của các hệ thống IDS/IPS lưu vết trạng thái (stateful) là nguy cơ bị tấn công **Cạn kiệt trạng thái (State Exhaustion)**. Kẻ tấn công có thể giả mạo hàng triệu địa chỉ IP nguồn ngẫu nhiên để gửi gói tin qua NIDS. Nếu hệ thống tự động khởi tạo hàng đợi mới cho mỗi IP nguồn mà không có cơ chế dọn dẹp, bộ nhớ RAM của hệ thống sẽ nhanh chóng bị lấp đầy và dẫn đến sập toàn bộ NIDS (Denial of Service).

Để giải quyết vấn đề bảo mật này, hệ thống triển khai một thuật toán dọn dẹp theo phong cách LRU (Least Recently Used) định kỳ sau mỗi 1000 gói tin được xử lý:

- Hệ thống đặt giới hạn số lượng nguồn theo dõi tối đa là 10.000 địa chỉ nguồn thông qua biến `_max_tracked_sources`.
- Khi số lượng nguồn theo dõi vượt quá giới hạn này, hàm `_cleanup_tracked_sources` sẽ tiến hành sắp xếp các địa chỉ IP theo mốc thời gian hoạt động gần nhất của chúng.
- Hệ thống sau đó tự động giải phóng (xóa bỏ) một nửa số lượng địa chỉ nguồn ít hoạt động nhất (được ghi nhận mốc thời gian cũ nhất) để nhường chỗ cho các nguồn mới, đảm bảo dung lượng bộ nhớ RAM luôn nằm trong tầm kiểm soát an toàn.

Các luật tích hợp gồm:

Rule ID	Attack	Nhóm phát hiện	Điều kiện mặc định	Severity
RULE-001	Port Scan	Behavior-based	≥ 20 TCP destination ports từ một source trong 10 giây.	Medium
RULE-002	ICMP Ping Flood	Behavior-based	≥ 100 ICMP request packets trong 10 giây.	High
RULE-003	TCP SYN Flood	Behavior-based	≥ 100 TCP SYN packets without ACK trong 10 giây.	High
RULE-004	DNS Tunneling	Behavior/anomaly-based	≥ 5 DNS query đáng ngờ từ cùng source trong 30 giây.	Medium

Bảng 3.1: Các detection rules tích hợp trong prototype

Với port scan, prototype chỉ đếm các probe flags như SYN, FIN, NULL và Xmas. Xmas scan tương ứng FIN + PSH + URG. ACK scan có thể tạo false positive nếu không có state tracking đầy đủ, vì vậy phần phát hiện ACK scan chỉ nên xem là minh họa trong prototype.

3.5 Luật chữ ký dạng Suricata-style subset

Ngoài luật hành vi, hệ thống hỗ trợ một tập con cú pháp kiểu Suricata/Snort. Người dùng có thể truyền tệp luật qua tham số `--rules`. Trong lần thử nghiệm được lưu ở `data/alerts.jsonl`, luật chữ ký được kích hoạt là luật DNS kiểm tra truy vấn chứa `example.com`:

Đoạn mã 3.5: Luật chữ ký dùng trong demo

```
1 alert dns any any -> any 53 (msg:"Custom DNS Query: example.com"; dns.query; content
  : "example.com"; nocase; classtype:policy-violation; priority:3; sid:1000101; rev
  :1;)
```

Mục tiêu của phần này là minh họa cách một signature engine hoạt động: phân tích header luật, chọn buffer cần kiểm tra, sau đó so khớp nội dung hoặc biểu thức chính quy. Đây là *Suricata-style subset*, không phải engine tương thích đầy đủ với Suricata/Snort.

Subset prototype hỗ trợ các thành phần cơ bản như header luật, `content`, `pcre`, `nocase`, một số sticky buffer DNS/payload và threshold đơn giản. Các tính năng chưa hỗ trợ đầy đủ gồm `depth/offset`, `distance/within`, `byte_test`, `flowbits`, TCP stream inspection, `file_data` và tối ưu `fast_pattern`. Tùy chọn flow hiện chỉ được kiểm tra ở mức hướng cơ bản, chưa phải stateful TCP flow tracking như IDS production.

3.6 Lưu trữ và Dashboard hiển thị

3.6.1 Lưu trữ cảnh báo dạng Append-Only JSON Lines (JSONL)

Hệ thống sử dụng định dạng tệp JSON Lines (JSONL) thông qua lớp `AlertStore` để lưu trữ nhật ký cảnh báo. Mỗi cảnh báo mới xuất hiện sẽ được mã hóa thành một chuỗi JSON độc lập và kết thúc bằng ký tự xuống dòng (`\n`). Quyết định thiết kế này dựa trên các đặc tính kỹ thuật quan trọng liên quan đến hiệu năng hệ thống:

- **Độ phức tạp ghi tối ưu $O(1)$:** Đối với định dạng JSON chuẩn (trong đó toàn bộ file là một mảng lớn `[]`), mỗi khi có cảnh báo mới, hệ thống bắt buộc phải đọc toàn bộ nội dung file cũ vào bộ nhớ RAM, phân tích cú pháp sang kiểu dữ liệu danh sách của Python, thêm phần tử mới, chuyển đổi lại thành chuỗi JSON và ghi đè toàn bộ tệp tin lên đĩa cứng. Thao tác này có độ phức tạp thời gian tăng dần theo độ lớn của lịch sử cảnh báo $O(M)$. Ngược lại, đối với JSONL, hệ thống chỉ cần mở tệp ở chế độ ghi tiếp (append mode - "a") và ghi trực tiếp dòng mới vào cuối tệp tin mà không cần quan tâm đến kích thước hiện tại của tệp. Thao tác này luôn đạt độ phức tạp $O(1)$.
- **Độ tin cậy và Khôi phục:** Trong trường hợp hệ thống NIDS bị sập đột ngột (ví dụ mất điện, lỗi crash), các dòng cảnh báo đã ghi trước đó vẫn hoàn toàn nguyên vẹn và không bị lỗi cấu trúc tệp tin (corrupted file) như khi ghi tệp tin JSON thông thường (vốn dễ bị thiếu dấu ngoặc đóng cuối tệp).
- **Tương thích hệ thống lớn:** Định dạng JSONL là định dạng chuẩn được hỗ trợ mặc định bởi các tác nhân thu thập log nổi tiếng như Filebeat, Fluentd hoặc Logstash để đẩy dữ liệu về hệ thống SIEM tập trung.

3.6.2 Cơ chế hiển thị trên Dashboard Flask

Dashboard được xây dựng dựa trên framework Flask (`dashboard/app.py`). Để đảm bảo Dashboard luôn hiển thị thông tin trực quan mà không tạo ra gánh nặng I/O đọc tệp tin liên tục cho ổ cứng máy chủ, hệ thống áp dụng cơ chế tối ưu hóa sau:

- **Cache theo mốc thời gian sửa đổi (File stat):** Trước khi thực hiện đọc và phân tích tệp tin `alerts.jsonl`, ứng dụng web kiểm tra thuộc tính mốc thời gian sửa đổi cuối cùng của tệp tin (`os.path.getmtime`). Nếu tệp tin không có thay đổi so với lần đọc trước, Dashboard sẽ trả về ngay kết quả đã được cache sẵn trong bộ nhớ RAM mà không cần thực hiện thao tác I/O đọc ổ cứng.
- **Phân tích thống kê thời gian thực:** Khi phát hiện tệp tin cảnh báo có sự thay đổi, Dashboard thực hiện đọc tệp, trích xuất dữ liệu để tính toán nhanh:
 - Tổng số lượng cảnh báo đã kích hoạt.
 - Biểu đồ thống kê số lượng cảnh báo phân nhóm theo mức độ nghiêm trọng (**severity**: High, Medium, Low).
 - Phân tích danh sách các địa chỉ IP nguồn xuất hiện nhiều nhất (**Top Attacker IPs**) hỗ trợ khoanh vùng mối đe dọa.
 - Danh sách hiển thị chi tiết mười cảnh báo gần nhất bao gồm đầy đủ bằng chứng (**evidence**) thu thập được từ gói tin.

Dashboard phục vụ quan sát và trình diễn kết quả; phần phản ứng tự động như chặn IP hoặc cập nhật tường lửa chưa nằm trong phạm vi bài tập.

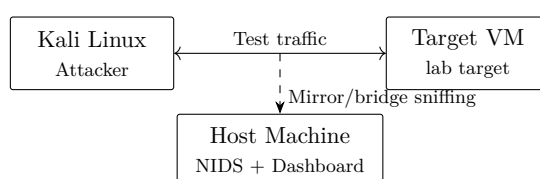
Chương 4

Kịch bản Thử nghiệm và Đánh giá

4.1 Môi trường thử nghiệm

Việc kiểm thử được thực hiện trong môi trường lab ảo hóa cô lập để tránh ảnh hưởng đến mạng thật. Các lệnh tạo traffic chỉ dùng trong lab/VM có kiểm soát và không chạy trên mạng thật nếu không có quyền. Mô hình gồm ba thành phần:

- **Host Machine:** chạy NIDS engine và Flask dashboard.
- **Kali Linux VM:** đóng vai trò máy tạo lưu lượng kiểm thử bằng `nmap`, `hping3` và `dig`.
- **Target VM:** nhận lưu lượng scan, flood và DNS trong môi trường lab.



Hình 4.1: Mô hình lab kiểm thử

Trong mô hình này, NIDS sniff traffic trên interface mirror/bridge của lab. Nếu dùng SPAN/TAP hoặc bridge trong môi trường thật, cần lưu ý khả năng mất packet hoặc asymmetric routing làm giảm độ đầy đủ của dữ liệu quan sát.

4.2 Khởi động hệ thống

Trên host, dashboard được chạy ở một terminal:

Đoạn mã 4.1: Khởi động dashboard

```
1 python3 dashboard/app.py
```

NIDS engine được chạy ở terminal khác. Interface cần thay bằng interface thực tế kết nối các máy ảo:

Đoạn mã 4.2: Khởi động NIDS engine

```
1 sudo .venv/bin/python main.py --iface <YOUR_INTERFACE> --rules custom_test.rules --limit 0
```

Khi dùng PCAP thay cho live capture, có thể chạy:

Đoạn mã 4.3: Phân tích PCAP

```
1 python3 main.py --pcap sample.pcap --rules custom_test.rules
```

4.3 Các kịch bản kiểm thử

4.3.1 Quét cổng bằng nmap

Kịch bản sử dụng **nmap** để gửi SYN tới nhiều cổng đích trong thời gian ngắn:

```
1 nmap -sS -p 21,22,23,25,53,80,110,135,139,143,443,445,\
2 993,995,1433,1723,3306,3389,5900,8080,8443,8888,9090,\
3 9200,9300 --scan-delay 30ms --max-retries 1 -T3 -n --open <TARGET_IP>
```

Kết quả mong đợi là cảnh báo **RULE-001 Port Scan**, vì cùng một IP nguồn truy cập nhiều cổng TCP đích trong cửa sổ 10 giây.

4.3.2 ICMP Ping Flood

Kịch bản gửi nhiều gói ICMP Echo Request:

```
1 hping3 --icmp -c 120 -i u80000 <TARGET_IP>
```

Tùy chọn **-i u80000** tạo khoảng cách 80000 micro giây giữa các gói, giúp lưu lượng đỡ đột ngột nhưng vẫn đủ vượt ngưỡng demo ≥ 100 packets/10 giây trong lab. Kết quả mong đợi là cảnh báo **RULE-002 ICMP Ping Flood** khi bộ đếm ICMP request từ cùng nguồn vượt ngưỡng.

4.3.3 TCP SYN Flood

Kịch bản gửi nhiều gói SYN tới cổng 80:

```
1 hping3 -S -p 80 -c 120 -i u80000 <TARGET_IP>
```

Tùy chọn **-S** tạo TCP SYN without ACK, còn **-i u80000** giữ tốc độ ổn định hơn nhưng vẫn đủ cao cho threshold demo. Kết quả mong đợi là cảnh báo **RULE-003 TCP SYN Flood**. Trong phạm vi bài tập, hệ thống chỉ cảnh báo dựa trên số lượng gói SYN without ACK từ cùng nguồn, chưa kiểm tra trạng thái hoàn tất kết nối ở mức TCP stream đầy đủ.

4.3.4 DNS tunneling

Kịch bản DNS tạo các truy vấn có nhãn dài/entropy cao. Để kích hoạt cảnh báo, truy vấn cần được lặp lại ít nhất 5 lần:

```
1 for i in {1..5}; do
2   dig "aGVsbG93b3JsZGh1bGxvd2...$i.tunnel.example.com"
3 done
```

Kết quả mong đợi là **RULE-004 DNS Tunneling Suspicion** khi có đủ số truy vấn bất thường trong cửa sổ thời gian.

Rule DNS tunneling có thể false positive với một số domain hợp lệ như CDN, tracking, DKIM, ACME challenge hoặc service discovery. Trong thực tế nên kết hợp thêm whitelist, NXDOMAIN rate, số lượng unique subdomain, qtype và reputation của base domain.

4.3.5 DNS signature rule

Kịch bản DNS bình thường và DNS tunneling cũng được dùng để kiểm tra luật chữ ký tùy chỉnh. Luật trong file `custom_test.rules` so khớp các truy vấn DNS chứa chuỗi `example.com` (chỉ là domain demo, không phải độc hại):

```
1 dig +short example.com @8.8.8.8
2 dig +short <LONG_LABEL>.tunnel.example.com @8.8.8.8
```

Kết quả mong đợi là cảnh báo signature-based **1000101 Custom DNS Query: example.com**. Luật này dùng `dns.query` và `content` để minh họa cơ chế nạp rule tùy chỉnh dạng *Suricata-style subset*.

4.4 Phân tích cấu trúc nhật ký cảnh báo sinh ra (JSONL)

Dưới đây là cấu trúc chi tiết của các bản ghi cảnh báo (Alert) được NIDS tự động phát hiện và xuất ra tệp `alerts.jsonl`. Việc phân tích cấu trúc này giúp các kỹ sư an toàn thông tin hiểu rõ cơ chế thu thập chứng cứ (evidence collection) của hệ thống:

4.4.1 Cảnh báo hành vi: Quét cổng (Port Scan)

Bản ghi cảnh báo dưới đây minh họa kết quả khi phát hiện máy Kali Linux (192.168.100.15) quét cổng mục tiêu:

Đoạn mã 4.4: Bản ghi JSONL cảnh báo Port Scan

```
1 {
2   "timestamp": "2026-06-24T05:00:12.145Z",
3   "rule_id": "RULE-001",
4   "attack_type": "Port Scan",
5   "detection_method": "behavior",
6   "severity": "Medium",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "192.168.100.20",
9   "description": "192.168.100.15 contacted 20 unique TCP ports in 10 seconds",
10  "evidence": {
11    "count": 20,
12    "unique_ports": [21, 22, 23, 25, 53, 80, 110, 135, 139, 143, 443, 445, 993, 995,
13                      1433, 1723, 3306, 3389, 5900, 8080]
14  }
```

Trường `evidence` lưu giữ danh sách các cổng đã quét để phục vụ điều tra xem dịch vụ nhạy cảm nào của hệ thống có nguy cơ bị lộ lọt thông tin.

4.4.2 Cảnh báo hành vi: ICMP Ping Flood

Khi lưu lượng Ping từ một nguồn vượt quá 100 gói tin trong 10 giây, cảnh báo có dạng như sau:

Đoạn mã 4.5: Bản ghi JSONL cảnh báo ICMP Flood

```
1 {
2   "timestamp": "2026-06-24T05:01:45.321Z",
3   "rule_id": "RULE-002",
4   "attack_type": "ICMP Ping Flood",
5   "detection_method": "behavior",
6   "severity": "High",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "192.168.100.20",
```

```
9  "description": "192.168.100.15 sent 100 ICMP packets in 10 seconds",
10  "evidence": {
11    "icmp_packets": 100
12  }
13 }
```

Cảnh báo này có mức độ nghiêm trọng **High** vì nó đe dọa trực tiếp tới tính sẵn sàng (Availability) của thiết bị mạng.

4.4.3 Cảnh báo hành vi: TCP SYN Flood

Cảnh báo SYN Flood ghi nhận hành vi flood cờ SYN mà không hoàn thành cái bắt tay ba bước:

Đoạn mã 4.6: Bản ghi JSONL cảnh báo TCP SYN Flood

```
1 {
2   "timestamp": "2026-06-24T05:03:10.987Z",
3   "rule_id": "RULE-003",
4   "attack_type": "TCP SYN Flood",
5   "detection_method": "behavior",
6   "severity": "High",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "192.168.100.20",
9   "description": "192.168.100.15 sent 100 SYN packets without ACK in 10 seconds",
10  "evidence": {
11    "dst_ip": "192.168.100.20",
12    "dst_port": 80,
13    "service_key": "192.168.100.15:192.168.100.20:80",
14    "syn_packets": 100,
15    "window_seconds": 10
16  }
17 }
```

Thông tin chi tiết về cổng đích (`dst_port = 80`) chỉ ra dịch vụ web đang là mục tiêu bị khai thác.

4.4.4 Cảnh báo bất thường: DNS Tunneling

Khi kẻ tấn công sử dụng công cụ tạo luồng đường truyền qua DNS, hệ thống phân tích Shannon Entropy và sinh cảnh báo:

Đoạn mã 4.7: Bản ghi JSONL cảnh báo DNS Tunneling

```
1 {
2   "timestamp": "2026-06-24T05:05:22.012Z",
3   "rule_id": "RULE-004",
4   "attack_type": "DNS Tunneling Suspicion",
5   "detection_method": "anomaly",
6   "severity": "Medium",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "8.8.8.8",
9   "description": "192.168.100.15 sent 5 DNS queries with tunneling indicators in 30 seconds",
10  "evidence": {
11    "latest_query": "ahZsbG93b3JsZGh1bGxvd29ybGQ.tunnel.example.com",
12    "max_label_entropy": 4.276,
13    "max_label_length": 54,
14    "query_length": 65,
15    "reason_counts": {
16      "high_entropy_label": 5,
17      "long_label": 5,
```

```
18     "long_query_name": 5
19   },
20   "recent_queries": [
21     "aGVsbG93b3JsZGh1bGxvd2...1.tunnel.example.com",
22     "aGVsbG93b3JsZGh1bGxvd2...2.tunnel.example.com",
23     "aGVsbG93b3JsZGh1bGxvd2...3.tunnel.example.com",
24     "aGVsbG93b3JsZGh1bGxvd2...4.tunnel.example.com",
25     "aGVsbG93b3JsZGh1bGxvd2...5.tunnel.example.com"
26   ],
27   "suspicious_queries": 5
28 }
29 }
```

Trường `max_label_entropy` và `reason_counts` là bằng chứng toán học đặc lực giúp phân loại hành vi tunneling thay vì các truy vấn DNS hợp lệ bình thường.

4.5 Đánh giá kết quả

Trong môi trường lab, các kịch bản trên kiểm tra được các nhóm phát hiện chính:

- Luật hành vi phát hiện các mẫu dựa trên tần suất và số lượng sự kiện trong thời gian ngắn.
- Luật signature phát hiện chuỗi đặc trưng trong truy vấn DNS.
- Dashboard hiển thị được số lượng cảnh báo, mức độ nghiêm trọng, IP nguồn và danh sách cảnh báo gần nhất.

Số liệu dưới đây được lấy trực tiếp từ file `data/alerts.jsonl`. File này lưu 14 cảnh báo thực nghiệm trong lần chạy lab: 3 cảnh báo behavior, 1 cảnh báo anomaly và 10 cảnh báo signature DNS.

Scenario	Expected alert	Observed alert	Evidence chính	Ghi chú
Port scan	RULE-001	1 alert	unique_ports ≥ 20, count = 20	Vượt ngưỡng scan trong cửa sổ 10 giây.
ICMP flood	RULE-002	1 alert	icmp packets = 100	Trigger đúng ngưỡng demo.
SYN flood	RULE-003	1 alert	syn packets = 100, dst_port = 80	SYN without ACK.
DNS tunneling	RULE-004	1 alert	suspicious queries = 5, max label entropy = 4.276, max label length = 54	≥ 5 suspicious queries/30 giây.
DNS signature	1000101	10 alerts	dns.query chứa example.com (domain demo)	Gồm truy vấn nền và các truy vấn tunnel.example.com.

Bảng 4.1: Bảng kết quả thực nghiệm từ file alert đã lưu

Kết quả này xác nhận hệ thống đáp ứng mục tiêu học tập và trình diễn. Tuy nhiên, đánh giá này chỉ có giá trị trong phạm vi lab có kiểm soát. Để kết luận về độ chính xác trong mạng thực tế cần có bộ dữ liệu lớn hơn, lưu lượng nền đa dạng hơn, kiểm thử false positive/false negative và đo hiệu năng xử lý gói tin.

Chương 5

Kết luận và Hướng phát triển

5.1 Kết quả đạt được

Bài tập đã xây dựng được một hệ thống NIDS nhỏ gọn phục vụ học tập và trình diễn. Hệ thống hoàn chỉnh các thành phần chính của một pipeline phát hiện xâm nhập mạng:

- Đọc lưu lượng từ PCAP hoặc live interface bằng Scapy.
- Chuẩn hóa packet thành cấu trúc `PacketEvent`.
- Phát hiện hành vi/anomaly bằng cửa sổ trượt thời gian với các ngưỡng lab/demo.
- Phân tích một *Suricata-style subset* để phát hiện chữ ký trong HTTP, DNS và payload.
- Tách domain blacklist khỏi nhóm rule mặc định; nếu cần phát hiện domain cụ thể thì nạp bằng luật chữ ký tùy chỉnh.
- Ghi cảnh báo ra JSON Lines và hiển thị qua dashboard Flask.
- Có kịch bản demo để kiểm tra các rule chính trong môi trường lab.

Kết quả quan trọng nhất của bài tập lớn là làm rõ cách các thành phần IDS phối hợp với nhau: capture cung cấp dữ liệu, parser chuẩn hóa dữ liệu, rule engine tạo cảnh báo, storage lưu lại kết quả và dashboard hỗ trợ quan sát.

5.2 Hạn chế

Hệ thống hiện tại vẫn có một số hạn chế kỹ thuật:

- **Chỉ phát hiện, chưa ngăn chặn:** hệ thống không chặn gói tin, không reset kết nối và không tự động cập nhật tường lửa.
- **Chưa xử lý lưu lượng mã hóa:** nội dung HTTPS/TLS không thể được kiểm tra bằng chữ ký nếu không có cơ chế giải mã hợp lệ.
- **Chưa tái tạo TCP stream đầy đủ:** payload bị chia nhỏ qua nhiều gói tin có thể làm giảm hiệu quả của signature matching.
- **Chưa xử lý đầy đủ fragmentation/segmentation:** IP fragmentation, TCP segmentation và out-of-order packets có thể làm sai lệch dữ liệu đầu vào của rule engine.
- **Dễ bị evasion với payload biến đổi:** payload bị encode, obfuscate hoặc chia nhỏ có thể vượt qua rule content đơn giản.

- **Phụ thuộc visibility của điểm giám sát:** SPAN/TAP có thể mất packet hoặc không nhìn thấy đủ hai chiều lưu lượng khi có asymmetric routing.
- **Hiệu năng còn giới hạn:** Python và Scapy phù hợp cho demo, nhưng chưa phù hợp với môi trường có lưu lượng lớn nếu không tối ưu thêm.
- **Phụ thuộc vào ngưỡng và luật:** chất lượng cảnh báo phụ thuộc vào cấu hình threshold, baseline từng mạng và độ đầy đủ của tập luật chữ ký.

5.3 Hướng phát triển

Các hướng cải tiến phù hợp với kiến trúc hiện tại gồm:

- Bổ sung IP defragmentation và TCP stream reassembly để kiểm tra payload ở mức phiên thay vì từng gói riêng lẻ.
- Bổ sung stateful protocol analysis cho TCP/HTTP/DNS và mở rộng parser rule theo hướng Snort/Suricata-like subset rõ ràng hơn.
- Thêm whitelist/baseline học từ traffic bình thường để giảm false positive.
- Thêm correlation engine để gom nhiều alert như port scan, flood hoặc HTTP SQLi thành incident có timeline.
- Thêm cơ chế cấu hình rule và threshold qua tệp YAML hoặc giao diện dashboard.
- Đánh giá trên PCAP lớn hoặc dataset công khai và báo cáo precision, recall, false positive rate.
- Đo hiệu năng trên PCAP lớn và tối ưu luồng xử lý để giảm packet drop.
- Tích hợp phản ứng bán tự động như xuất IOC, webhook hoặc cập nhật danh sách chặn cho firewall trong môi trường lab.
- Xây dựng bộ kiểm thử với cả traffic bình thường và traffic tấn công để đánh giá false positive/false negative rõ ràng hơn.

5.4 Kết luận

Hệ thống đã đáp ứng mục tiêu chính của bài tập lớn: minh họa được một NIDS hoạt động từ khâu thu thập gói tin đến sinh cảnh báo và trực quan hóa kết quả. Dù chưa phải hệ thống production, bài tập cung cấp nền tảng thực hành tốt để hiểu các kỹ thuật phát hiện xâm nhập mạng và các giới hạn khi triển khai IDS trong thực tế.